Week 2 →

Coarse Coding :→

The features used to construct value estimates are one of the most important parts of the RL agent.

∅ In state aggregation, we associate nearby state with same feature. In general, we can aggregate states using any shapes we want as long as those shapes do not have any gaps or overlap.

∅ State aggregation generally does not allow the shapes to overlap.

∅ We obtain more flexible class of feature representations called Coarse Coding.

Therefore Coarse Coding group states into features of arbitrary shapes and sizes.

⇒ Note that performing an update to the weights in one state changes the value estimate for all states within the receptive fields of active features.

⇒ If the Union of receptive fields for the active features is large, the feature representation generalises more!

⇒ However, if the Union is small, there is little generalisation!

⇒ Therefore, shape and size of the receptive fields impact generalization and so the speed of the learning.

In Coarse Coding, the overlap b/w the circles (any shape in general) dictates the level of discrimination.

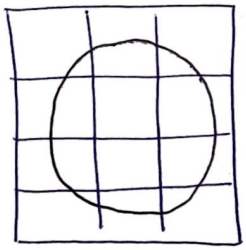
⇒ Example!

- Consider the learning approximation to a step function! let's assume we can sample the true function values in order to estimate

- Video!

Tile Coding :→

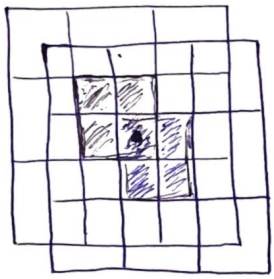
- It is the specific type of Coarse Coding.
- It performs an exhaustive partition of the state space using overlapping grids.
- Unlike Coarse Coding, tile coding uses squares. This grid (bunch of squares) is called tile.



→ tilling

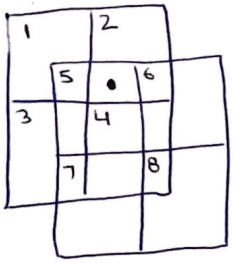
- larger tiles will increase the generalisation.
- To improve the discriminative ability of our tile coding, we can put several tillings on top of each other. Each tilling is offset by small amount.

Offsetting each layer of tilling creates many small intersections. This results in better discrimination



- In practice, it is useful to use a large number of tillings.
- For one tilling, generalisation only occurs within the square, but with multiple tillings, updates in this state generalises to other states.

⇒ The number of active tiles is always significantly less than the number of total tiles.



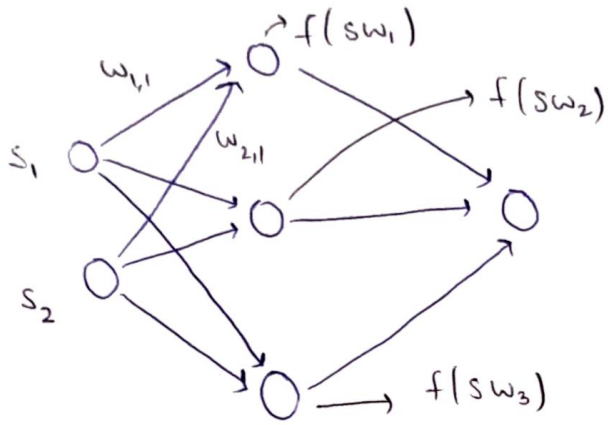
$x(s) = [0\ 1\ 0\ 0\ 1\ 0\ 0\ 0]$

1 2 3 4 5 6 7 8

↑ feature vector for state (as shown with dot)

Tile Coding is effectively multiple instances of state aggregation or tillings layered on top of each other.

Neural Network →



Input layer | Hidden layer | Output layer

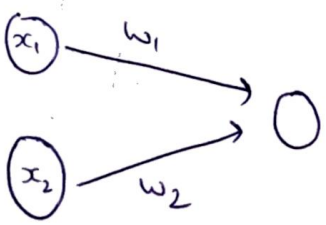
$S = [s_1, s_2]$ $\quad w_1 = [w_{1,1} \; w_{2,1}]^T$

$$W = [w_1 \; w_2 \; w_3] = \begin{bmatrix} w_{1,1} & w_{2,1} & w_{3,1} \\ w_{1,2} & w_{2,2} & w_{3,2} \end{bmatrix}$$

Outputs (for 2nd layer) $= f(sw)$

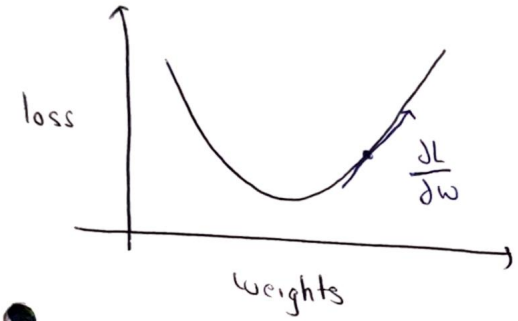
Ø let's first initialize the weights as

$$w_{init} \sim N(\mu, \sigma)$$



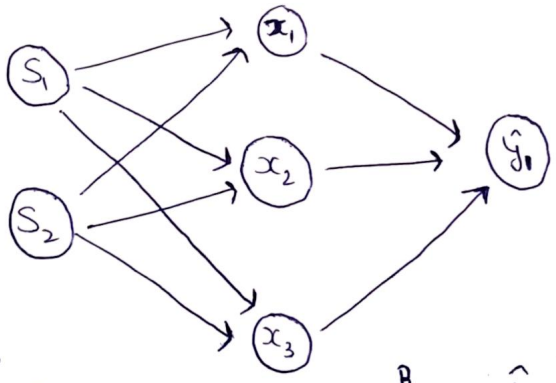
$\Rightarrow \quad f(w_1 x_1 + w_2 x_2)$

{ non-linear activation function! }



$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

- The gradient of a function points in the direction of greatest ascent.
- By moving in the direction opposite the gradient, we move in the direction that most quickly minimizes the loss.



{ S A x B $\hat{y}$ }

Loss : $L(\hat{y}_k, y_k) = (\hat{y}_k - y_k)^2$

Derivation of the gradient :-

1) $\dfrac{\partial L(\hat{y}_k, y_k)}{\partial B_{jk}}$ $\qquad x = f_A(SA)$

$\qquad\qquad\qquad\qquad\qquad\qquad\quad \hat{y} = f_B(xB)$

$$= \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial B_{jk}} \qquad \left.\begin{array}{l}\\\\\end{array}\right\} \; \theta = xB$$

$$= \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot \frac{\partial (f_B(xB)_k)}{\partial B_{jk}}$$

$$= \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot \frac{\partial f_B(\theta_k)}{\partial B_{jk}}$$

$$= \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot \frac{\partial f_B(\theta_k)}{\partial \theta_k} \cdot \frac{\partial \theta_k}{\partial B_{jk}}$$

(θ is linear function of B.

$$= \left\{\frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k}\right\} \cdot \left\{\frac{\partial f_B(\theta_k)}{\partial \theta_k}\right\} \cdot \{x_j\} \quad \Rightarrow$$

(and also x doesn't depend on B.

let $\quad L = \frac{1}{2}(\hat{y}_k - y_k)^2$

and $\quad f_B(\theta_k) = \theta_k \quad \Rightarrow$ linear activation on the last layer.

Therefore we have,

$$= (\hat{y}_k - y_k) \times 1 \times x_j$$

$$= x_j(\hat{y}_k - y_k)$$

---

also let, $\delta_k^B = \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot \frac{\partial f_B(\theta_k)}{\theta_k}$

thus, $= \delta_k^B x_j$

⇒ 2) $\frac{\partial L(\hat{y}_k, y_k)}{\partial A_{ij}} = \delta_k^B \frac{\partial \theta_k}{\partial A_{ij}}$

$$= \delta_k^B \frac{\partial (x_j B_{jk})}{\partial A_{ij}} = \delta_k^B B_{jk} \frac{\partial x_j}{\partial A_{ij}} \Bigg\} \psi = sA$$

$$= \delta_k^B B_{jk} \frac{\partial (f_A(s_i A_{ij}))}{\partial A_{ij}}$$

$$= \delta_k^B B_{jk} \frac{\partial (f_A(\psi_j))}{\partial A_{ij}}$$

$$= \delta_k^B B_{jk} \left(\frac{\partial f_A(\psi_j)}{\partial \psi_j}\right)\left(\frac{\partial \psi_j}{\partial A_{ij}}\right)$$

$$= \underbrace{\delta_k^B B_{jk} \frac{\partial f_A(\psi_j)}{\partial \psi_j}}_{\delta_j^A} s_i$$

$$= \delta_j^A S_i$$

The Backprop algorithm $\longrightarrow$

for each (s, y) in D :

$$\delta_k^B = \frac{\partial L(\hat{y}_k, y_k)}{\partial \hat{y}_k} \frac{\partial f_B(\theta_k)}{\partial \theta_k}$$

$$\nabla_B^{jk} = \delta_k^B x_j$$

$$B = B - \alpha_B \nabla_B$$

$$\delta_j^A = \left(B_{jk} \delta_k^B \right) \frac{\partial f_A(\psi_j)}{\partial \psi_j}$$

$$\nabla_A^{ij} = \delta_j^A S_i$$

$$A = A - \alpha_A \nabla_A$$

$$v = x w^{(1)} + b^{(1)}$$

$$\frac{\partial v}{\partial w_{ij}^{(0)}} = \frac{\partial}{\partial w_{ij}^{(0)}} \left[x w^{(1)} + b^{(1)} \right]$$

$$= w_j^{(1)} \frac{\partial}{\partial w_{ij}^{(0)}} \left[x_i \right]$$

$$= w_j^{(1)} \frac{\partial}{\partial w_{ij}^{(0)}} \left[\max(0, \psi_i) \right]$$

$$= w_j^{(1)} \frac{\partial}{\partial w_{ij}^{(0)}} \left[S_i w_{ij}^{(0)} + b^{(0)} \right]$$

$$= w_j^{(1)} S_i$$